

SEMINAR

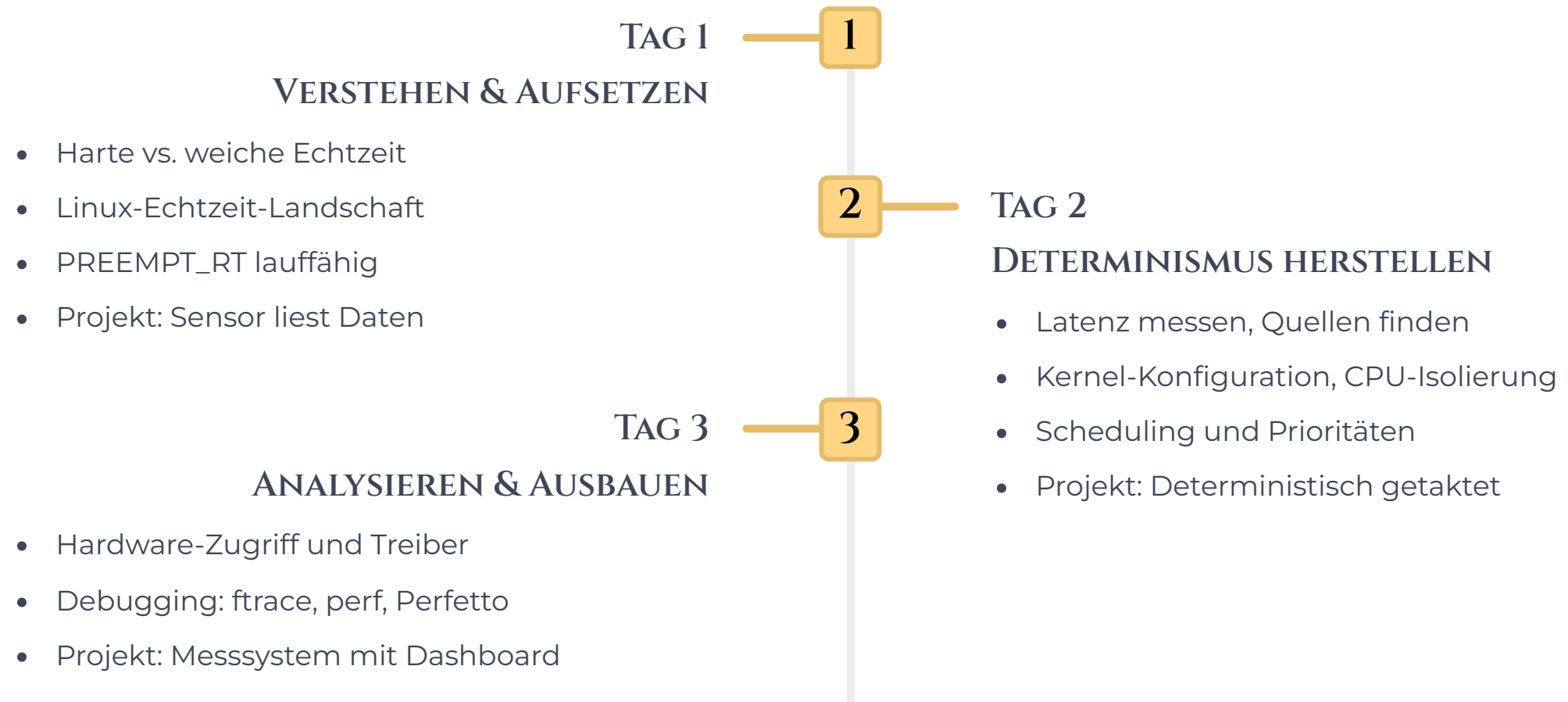
ECHTZEIT-LINUX

GRUNDLAGEN UND PRAKTISCHE ANWENDUNGEN

Determinismus, Latenz und der Weg zum RT-Kernel

DIE ECHTZEIT-LANDKARTE

WO WIR IN DREI TAGEN HINLAUFEN



ECHTZEIT HEISST NICHT SCHNELL

ECHTZEIT = RECHTZEITIG

Nicht schnell. Nicht maximaler Durchsatz. Rechtzeitig.

DURCHSATZ

Optimiert den Durchschnitt

ECHTZEIT

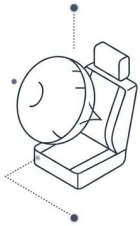
Optimiert den schlechtesten Fall

KONSEQUENZ

Ein langsames System kann echtzeitfähig sein

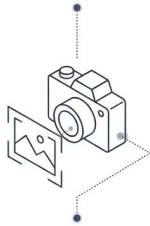
HART, FEST, WEICH

HART



HART:
Deadline
verpasst =
Systemausfall
(Beispiel:
Airbag)

FEST



FEST:
Ergebnis
wertlos,
System läuft
weiter
(Beispiel:
Videoframe)

WEICH



WEICH:
Wert sinkt
allmählich,
System läuft
weiter
(Beispiel:
Temperatur-
logger)

DREI KLASSEN, DREI KONSEQUENZEN

HART

Frist verletzt =
Systemversagen.
Airbag,
Motorsteuerung.

FEST

Ergebnis
wertlos, System
läuft weiter.
Videoframe.

WEICH

Nutzwert sinkt
allmählich.
Streaming,
Temperaturlogg
er.

DIE VIER GRÖSSEN



LATENZ

Zeit zwischen Ereignis und Reaktion



JITTER

Schwankung dieser Reaktionszeit



DETERMINISMUS

Worst Case ist bekannt und beschränkt



DURCHSATZ

Menge pro Zeit — hier **nicht** das Ziel



Jitter ist die Schwankung der Latenz — nicht die Latenz selbst. Der Worst Case, nicht der Mittelwert, entscheidet.

ECHTZEIT ODER NICHT?

HART, WEICH ODER GAR NICHT? — EINORDNEN OHNE AUFLÖSUNG

AIRBAG-AUSLÖSUNG

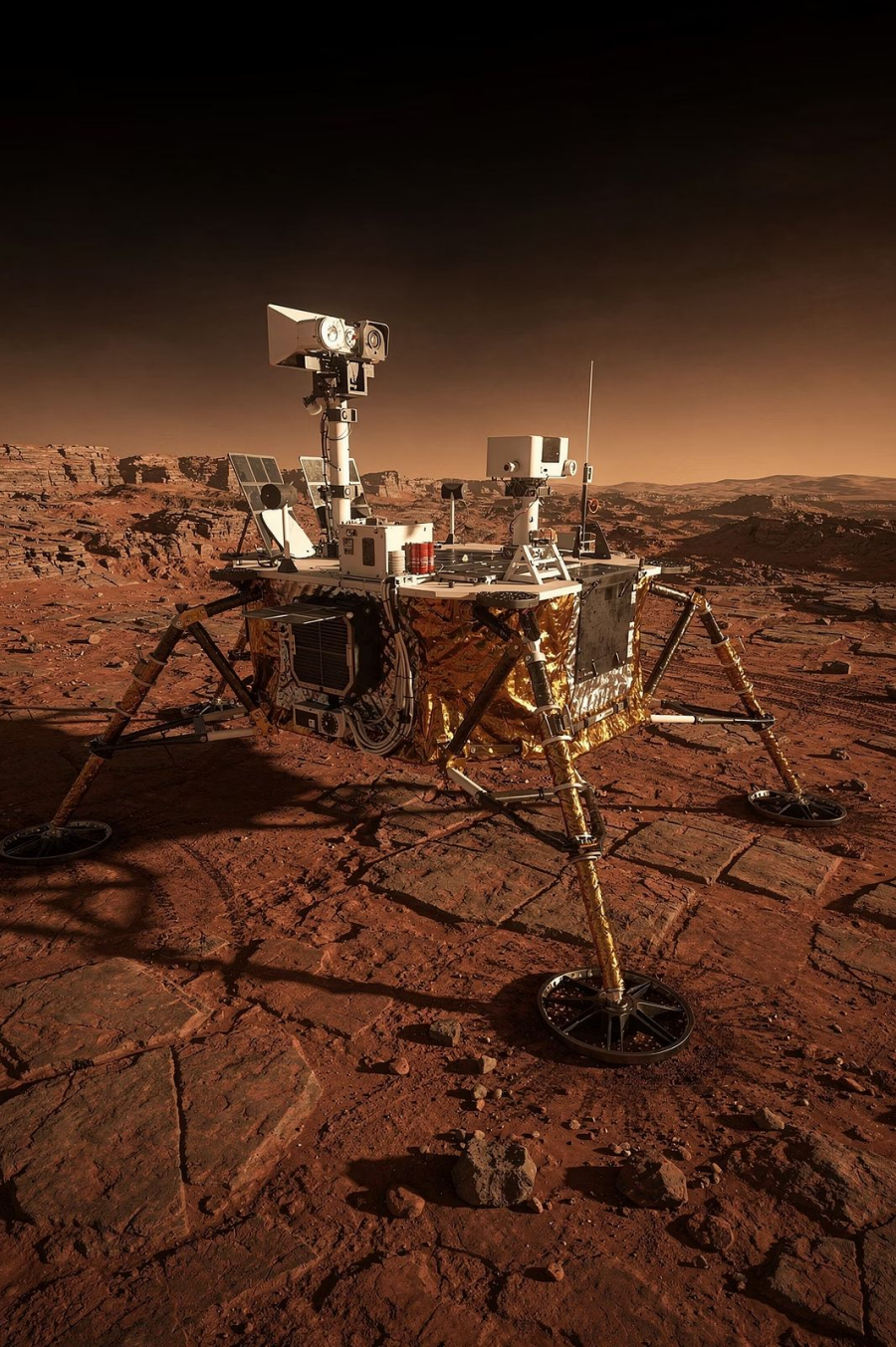
NETFLIX-STREAM

HERZSCHRITTMACHER

TEMPERATUR-LOGGER IM KÜHLTRANSPORT

ROBOTERARM-REGELUNG

BANKABRECHNUNG



4. JULI 1997

Nach sieben Monaten Flug landet Mars Pathfinder erfolgreich. Die NASA feiert. Die Bilder kommen. Alles funktioniert.

DANN FING ES AN. DER BORDCOMPUTER STARTETE SICH NEU.

Immer wieder. Ohne erkennbares Muster. Millionen Kilometer entfernt. Daten gingen verloren.



WAS WAR PASSIERT?

KEIN HARDWAREDEFEKT.

Kein Programmierfehler im klassischen Sinn. Ein Scheduling-Problem.

📅 — Auflösung morgen —



DER WIDERSPRUCH

Linux wurde gebaut für Maximaldurchsatz. Echtzeit braucht das Gegenteil.

LINUX WURDE GEBAUT FÜR

- Maximalen Durchsatz
- Fairness zwischen Prozessen
- Effiziente Ressourcennutzung
- Viele parallele Nutzer

ECHTZEIT BRAUCHT

- Garantierte Obergrenzen
- Strikte Priorisierung
- Reservierte Ressourcen
- Eine kritische Aufgabe

Und trotzdem läuft es in immer mehr Industrieanlagen.

WAS DAFÜR SPRICHT

EIN OS FÜR ALLES

Steuerung, Netzwerk und Visualisierung im selben System

KEIN LIZENZMODELL

Keine Kosten pro Gerät, riesiges Ökosystem

VOLLSTÄNDIGER NETZWERK-STACK

OPC-UA, PROFINET, MQTT — out of the box

WERKZEUGE, DIE RTOS NICHT KENNT

ftrace, perf, Valgrind, GDB — vollständig integriert

WAS DAGEGEN SPRICHT

LATENZEN

Mikrosekunden-Bereich — nicht Nanosekunden

RESSOURCENBEDARF

Größerer Speicher, längere Bootzeit

ZERTIFIZIERUNG

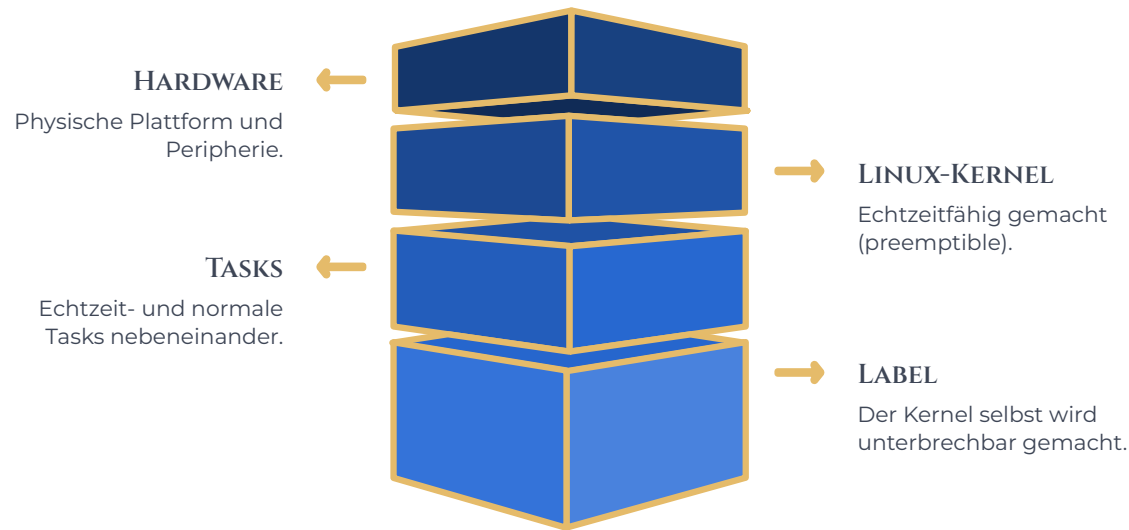
IEC 61508, DO-178C: deutlich aufwändiger

KOMPLEXITÄT

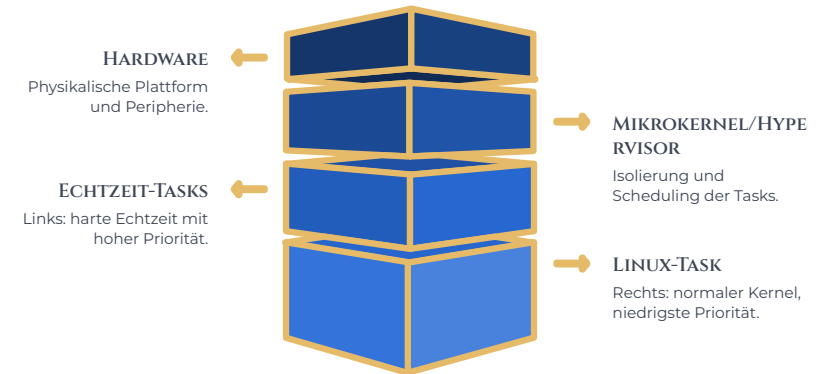
Mehr Abstraktionsschichten, mehr Fehlerpotential

ZWEI GRUNDSÄTZLICH VERSCHIEDENE WEGE

SINGLE-KERNEL: PREEMPT_RT



DUAL-KERNEL: XENOMAI / RTAI



WARUM PREEMPT_RT GEWONNEN HAT

MAINLINE SEIT 6.12

Kein Patchen mehr nötig — ab Kernel 6.12 integriert

STANDARD-API

POSIX, pthreads, Standard-Werkzeuge — keine Doppelwelt

EIN TREIBERMODELL

Ein Kernel, ein Subsystem, keine parallele Infrastruktur

UPSTREAM-PFLEGE

Wird vom Kernel-Team langfristig gewartet

WAS PREEMPT_RT TECHNISCH TUT

01

SPINLOCKS → SCHLAFENDE MUTEXE

Kernel-Locks können verdrängt werden

02

INTERRUPT-HANDLER ALS KERNEL-THREADS

IRQ-Handler laufen mit steuerbarer Priorität

03

PRIORITY INHERITANCE

Verhindert Prioritätsinversion systematisch

04

HOCHAUFLÖSENDE TIMER

hrtimer statt Tick-basiertem Scheduling

XENOMAI UND RTAI IN EINEM SATZ

DUAL-KERNEL-ANSATZ

Technisch niedrigere Latenzen möglich

EIGENE API

Portierung nötig — kein Standard-POSIX

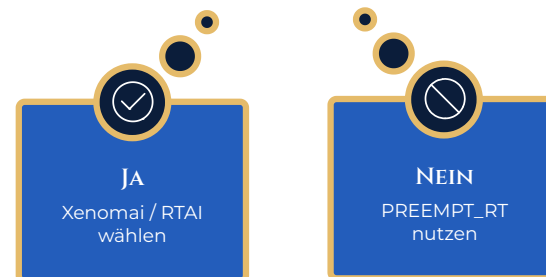
EIGENES TREIBERMODELL

Hardware-Support eingeschränkt

SINNVOLL BEI

Extremen Anforderungen im Nanosekundenbereich

EINORDNUNG:
Brauche ich Nanosekunden-Latenzen?



Einordnung. Kein Vertiefungsthema in diesem Kurs.

DAS EXPERIMENT

EINE HARDWARE, EINE SOFTWARE, EIN UNTERSCHIED

- Hardware: Raspberry Pi 4
- Unterschied: Preemption-Modell

VIER SZENARIEN

- Ohne Last
- CPU-Last
- Volle Last
- Isolierter Kern

☐ Welche Zahl entscheidet — Mittelwert oder Maximum?



NACHMITTAG

VOM ZUHÖREN ZUM BAUEN

AB JETZT TIPPEN ÜBERWIEGEND SIE.

VIER WEGE ZU EINEM RT-KERNEL

Weg	Aufwand	Einsatzgebiet	Hinweis
Fertiges Image nutzen	Minuten	Schulung, Prototyp	
defconfig-Build	30 Minuten	Standard seit Kernel 6.12	unser Weg
Yocto-Integration	Tage	Serienprodukt	
Manuelles RT-Patchen	historisch	nicht mehr nötig	veraltet

WORAN MAN EINEN RT-KERNEL ERKENNT

So überprüfen Sie schnell und zuverlässig, ob auf Ihrem System ein Echtzeit-Kernel aktiv ist:

uname -a

Die Ausgabe des Kernel-Namens sollte die Zeichenkette `PREEMPT_RT` enthalten.

KERNEL-KONFIGURATION PRÜFEN

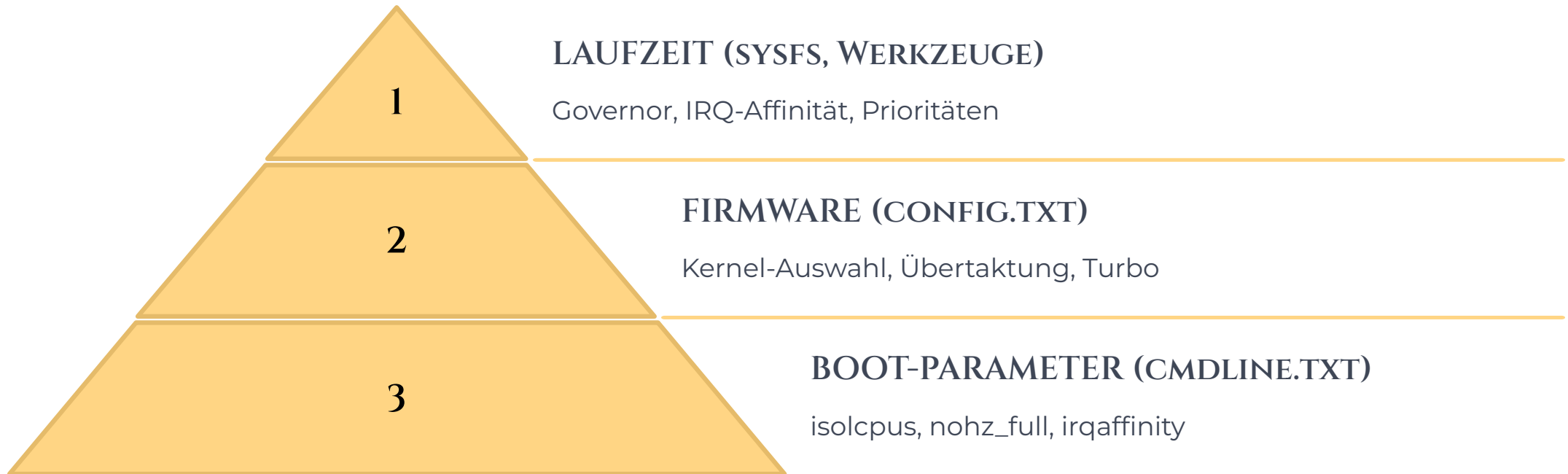
Überprüfen Sie in der Konfigurationsdatei des laufenden Kernels (z.B. `/boot/config-$(uname -r)`), ob die Option `CONFIG_PREEMPT_RT=y` gesetzt ist.

PROZESSLISTE ÜBERPRÜFEN

Interrupt-Handler (Prozesse wie `irq/*`) erscheinen als eigene, priorisierbare Threads in der Prozessliste (z.B. mit `ps -aux` oder `top`). Dies ist der stärkste und praktischste Indikator für einen `PREEMPT_RT`-Kernel.

Die letzte Prüfung ist die aussagekräftigste.

WO DIE SCHRAUBEN SITZEN



EIN ECHTZEITSYSTEM AUFSETZEN UND PRÜFEN

1

VORMACHEN

2

ERKLÄREN

3

NACHMACHEN

4

ÜBEN

LERNPROJEKT STUFE 1: DER SENSOR SPRICHT

ERST BAUEN WIR ES SO, WIE ES JEDER BAUT.

In dieser ersten Stufe setzen wir ein System auf, das einen Umgebungssensor ausliest und dessen Zeitverhalten prüft – zunächst ohne Echtzeit-Optimierungen.

- Auslesen des BME280 Sensors über I²C
- Zyklische Messung alle 100 Millisekunden
- Protokollierung und Analyse des Zeitverhaltens
- Implementierung noch ohne spezifische Echtzeit-Mechanismen

60 SEKUNDEN LAUFZEIT · 100 MS PERIODE

**WO SIND DIE FEHLENDEN
MESSUNGEN HIN?**

WAS HEUTE STAND

- 1 ECHTZEIT HEISST RECHTZEITIG, NICHT SCHNELL
- 2 DETERMINISMUS IST EINE AUSSAGE ÜBER DAS MAXIMUM
- 3 PREEMPT_RT IST SEIT KERNEL 6.12 IN MAINLINE
- 4 EIN SYSTEM OHNE LAST ZU MESSEN BEWEIST NICHTS
- 5 UNSER SENSOR LIEST — ABER DIE ZEITPUNKTE DRIFTEN



MORGEN

DIE DRIFTENDE SCHLEIFE REPARIEREN

Stabilisierung der Messintervalle
für präzise Datenerfassung.

LATENZQUELLEN FINDEN UND ABSTELLEN

Analyse und Eliminierung
unerwünschter Verzögerungen
im System.

CPU-KERNE ISOLIEREN, PRIORITÄTEN SETZEN

Optimierung der
Prozessorressourcen für
Echtzeitanwendungen.